



💎💎所属专栏: [Linux](#)💎💎

💎💎作者主页: [数某](#)💎💎

Linux: 线程同步与互斥

线程互斥

我们先明确几个概念

1. 临界资源: 多线程执行流共享的资源, 一个进程中所有线程都要访问的资源
2. 临界区: 每个线程内部, 访问临界资源的代码
3. 互斥: 任何时候, 互斥保证有且只有一个线程进入临界区执行访问临界资源的代码, 对临界资源起保护作用
4. 原子性: 不会被任何调度机制打断的操作, 只有两种状态, 要么完成了, 要么没完成, 不存在完成中、正在完成的状态

互斥量mutex

大部分情况, 线程使用的数据都是局部变量, 变量的地址都在线程地址空间内, 变量归属单个线程, 其他线程无法获得这种变量

但有些变量需要在线程间共享, 称为共享变量, 通过共享变量, 完成线程间的交互

```
1 #include "mutex.hpp"
```

```
2  #include <pthread.h>
3  #include <unistd.h>
4
5  int ticket = 10000;
6
7  void *routine(void *args)
8  {
9      Mutex *m = static_cast<Mutex *>(args);
10     while (true)
11     {
12         // m->Lock();
13         if (ticket > 0)
14         {
15             usleep(100);
16             std::cout << "抢票一次" << std::endl;
17             ticket--;
18             std::cout << "剩余票数: " << ticket <<
std::endl;
19             // m->UnLock();
20         }
21         else
22         {
23             // m->UnLock();
24             break;
25         }
26     }
27     return nullptr;
28 }
29
30 int main()
31 {
32     Mutex mutex;
33
34     pthread_t t1;
35     pthread_create(&t1, nullptr, routine, (void *)&mutex);
36     pthread_t t2;
37     pthread_create(&t2, nullptr, routine, (void *)&mutex);
38     pthread_t t3;
39     pthread_create(&t3, nullptr, routine, (void *)&mutex);
40     pthread_t t4;
41     pthread_create(&t4, nullptr, routine, (void *)&mutex);
42 }
```

```

43     while (true)
44     {
45         pthread_join(t1, nullptr);
46         pthread_join(t2, nullptr);
47         pthread_join(t3, nullptr);
48         pthread_join(t4, nullptr);
49     }
50
51     return 0;
52 }

```

注意：mutex.h头文件是提前封装好的

上面的代码，在没有加锁的情况下，会出现票售多了的情况。

if 语句判断条件为真后，代码可以并发的切换到其他线程，usleep 就是在模拟这个漫长的业务，此时票数还没有减减，另外可能又有几个线程就又售出了多张票

而且tick--本身就不是一个原子操作，**我们认为，一条汇编代码是原子的**

```

1  取出ticket--部分的汇编代码
2  objdump -d a.out > test.objdump
3  152 40064b: 8b 05 e3 04 20 00 mov 0x2004e3(%rip),%eax
   #600b34 <ticket>
4  153 400651: 83 e8 01 sub $0x1,%eax
5  154 400654: 89 05 da 04 20 00 mov %eax,0x2004da(%rip)
   #600b34 <ticket>

```

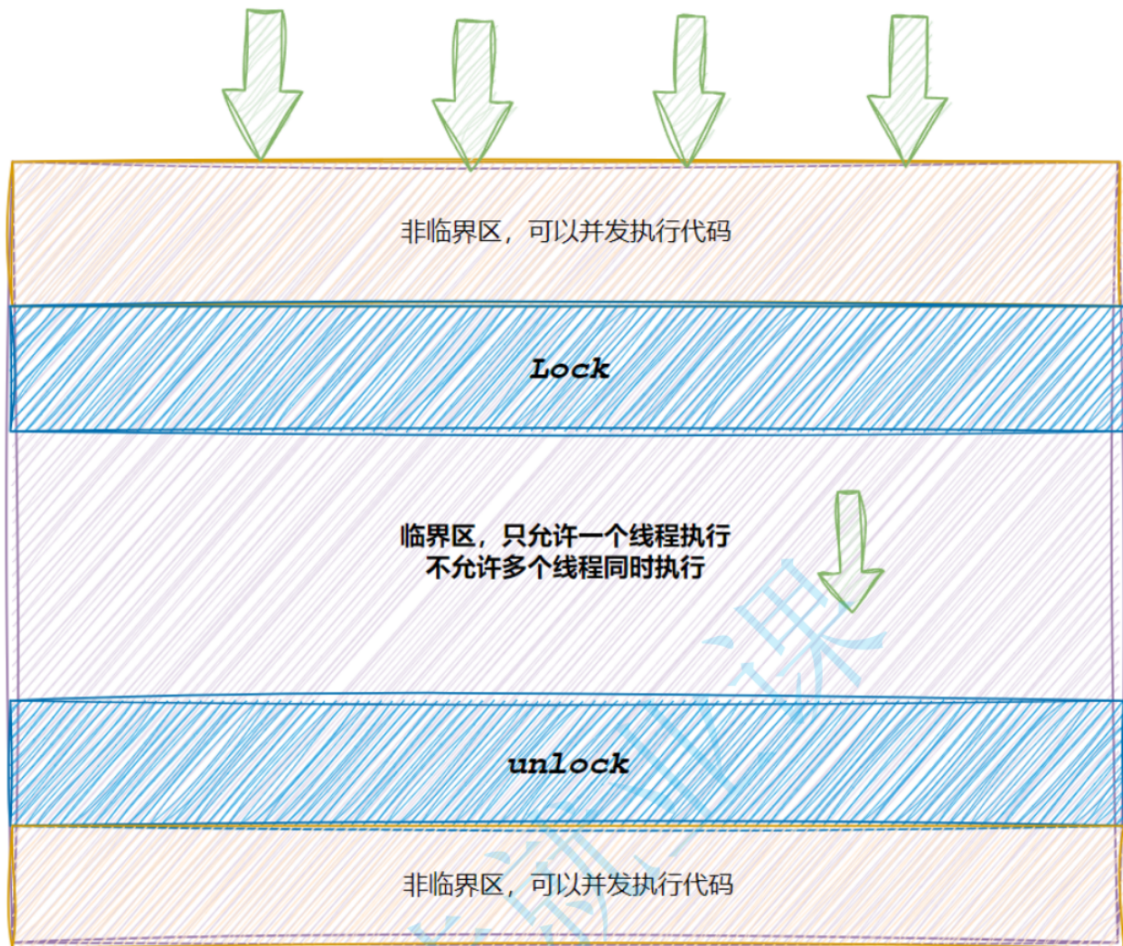
-- 操作并不是原子的，对应了三条汇编指令

- load：将共享变量ticket从内存加载到寄存器中
- update：更新寄存器里面的值，执行-1操作
- store：将新值，从寄存器写回共享变量ticket的内存地址

要解决以上问题，需要三点

- 代码必须要有互斥行为：当代码进入到临界区执行时，不允许其他线程进入该临界区。
- 如果多个线程同时要求执行临界区代码，并且临界区没有线程在执行，那么只能允许一个线程进入该临界区
- 如果线程不在临界区中执行，那么该线程不能阻止其他线程进入临界区

OK，这时候就需要锁出场了。Linux 中把这种锁叫做互斥量（互斥锁）



互斥量的接口

初始化互斥量

静态分配

```
1 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER
```

动态分配

```
1 int pthread_mutex_init(pthread_mutex_t *restrict mutex,  
  const pthread_mutexattr_t *restrict attr);  
2 参数:  
3     mutex: 要初始化的互斥量  
4     attr: NULL
```

销毁互斥量

注意:

使用 `PTHREAD_MUTEX_INITIALIZER` 不需要销毁

不要销毁一个已经加锁的互斥量（该互斥锁已经有线程在使用中了）

已经销毁的互斥量，要确保后面不会有线程再次尝试加锁

```
1 | int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

互斥量加锁和解锁

```
1 | int pthread_mutex_lock(pthread_mutex_t *mutex);  
2 | int pthread_mutex_unlock(pthread_mutex_t *mutex);  
3 |     返回值：成功返回0，失败返回错误号
```

调用 `pthread_mutex_lock` 时可能会遇到以下情况：

互斥量处于未锁状态，该函数会将互斥量锁定，同时返回0

发起函数调用时，其他线程已经锁定互斥量，或者存在其他线程同时申请互斥量，但没有竞争到互斥量，那么 `pthread_mutex_lock` 调用会陷入阻塞（执行流被挂起）等待互斥量解锁后重新申请

互斥量实现原理

现在我们知道，`i++` 和 `++i` 这种操作都不是原子的，会有数据一致性的问题。

为了实现互斥锁操作，大多数体系结构都提供了 `swap` 或者 `exchange` 指令，该指令的作用是把寄存器和内存单元的数据相交换，由于只有一条汇编指令，保证了原子性，即使是在多处理器平台上，访问内存的总线周期也有前后之分，一个处理器上的交换指令执行时另一个处理器的交换指令只能等待总线周期。现在我们把 `lock` 和 `unlock` 的伪代码改一下

```

1 lock:
2     movb $0, %a1
3     echgb %a1, mutex
4     if(a1寄存器内容 > 0) {
5         return 0;
6     } else
7         挂起等待;
8     goto lock;
9 unlock:
10    movb $1, mutex
11    唤醒等待Mutex的线程;
12    return 0;

```

互斥量的封装

```

1 // mutex.hpp
2 #pragma once
3 #include <iostream>
4 #include <unistd.h>
5 #include <pthread.h>
6 #include <mutex>
7
8 class Mutex
9 {
10 public:
11     // 还可以删除不需要的拷贝构造和赋值重载
12     Mutex()
13     {
14         pthread_mutex_init(&_amp;mutex, nullptr);
15     }
16     void Lock()
17     {
18         pthread_mutex_lock(&_amp;mutex);
19     }
20     void UnLock()
21     {
22         pthread_mutex_unlock(&_amp;mutex);
23     }
24     pthread_mutex_t *Get() // 得到原始指针
25     {
26         return &_amp;mutex;
27     }
28 };

```

```

27     }
28     ~Mutex()
29     {
30         pthread_mutex_destroy(&_mutex);
31     }
32
33 private:
34     pthread_mutex_t _mutex;
35 };
36
37 // RAII风格, 进行锁管理
38 class global_mutex
39 {
40 public:
41     global_mutex(Mutex &mutex)
42         : _mutex(mutex)
43     {
44         _mutex.Lock();
45     }
46     ~global_mutex()
47     {
48         _mutex.Unlock();
49     }
50 private:
51     Mutex &_mutex;
52 };

```

注意：带有 `pthread.h` 的源码编译时要链接 `pthread` 库

```

1 // 这里我们所做的封装是模仿`C++11`的
2 std::mutex mtx;
3 std::lock_guard<std::mutex> guard(mtx);

```

线程同步

条件变量

当一个线程互斥地访问一个变量时，它必须要等到其他线程先把该变量修改之后才访问，那这个时候这个线程在其他线程访问之前什么也做不了。

例如在一个线程访问队列时，发现队列为空，因为没有其他线程往队列里塞数据，只能等待，等到队列里被其他线程塞了数据之后它才访问队列，这种情况就需要访问队列的线程在条件变量下等待，在其他线程塞完数据后通知该线程，然后该线程被唤醒，访问队列。

同步概念与竞态条件

同步 Synchronization：在保证数据安全的前提下，让线程能够**按照某种特定的顺序**访问临界资源，从而有效避免饥饿问题

竞态条件 Race Condition：是多线程或多进程编程中因并发执行导致的一种错误，当多个线程或进程同时访问和操作共享资源，且最终结果依赖于执行时序时，就会出现竞态条件。

条件变量函数

初始化

```
1 int pthread_cond_init(pthread_cond_t *restrict cond,  
2   const pthread_condattr_t *restrict attr);  
3 参数：  
4      cond: 要初始化的条件变量  
5      attr: NULL
```

销毁

```
1 int pthread_cond_destroy(pthread_cond_t * cond)
```

等待条件满足

```
1 int pthread_cond_wait(pthread_cond_t *restrict cond,  
2   pthread_mutex_t *restrict mutex);  
3 参数：  
4      cond: 要在这个条件变量上等待  
5      mutex: 互斥量，等待时释放锁
```

唤醒等待

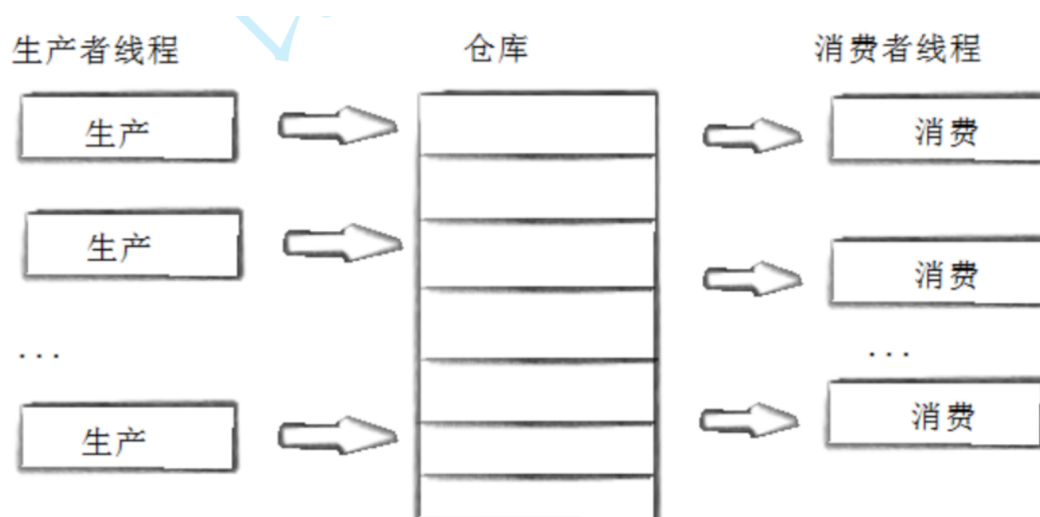
```
1 int pthread_cond_broadcast(pthread_cond_t *cond);  
2 int pthread_cond_signal(pthread_cond_t *cond);
```


生产消费者模型

321原则：三种关系，两种角色，一个交易场所

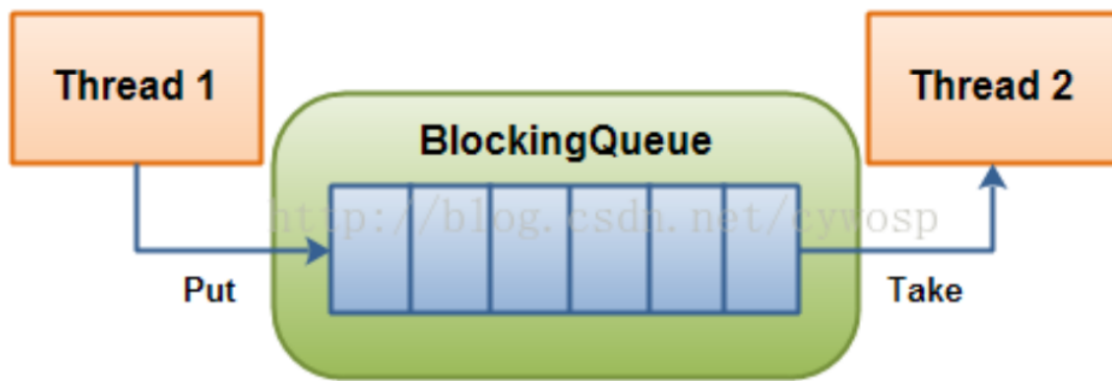
生产消费者模式是通过一个容器来解决生产者和消费者的强耦合问题。生产者和消费者彼此之间不直接通讯，通过中间的容器（如阻塞队列，循环队列）来通讯，所以生产者盛产完的数据不用等消费者处理，直接扔给容器，消费者也不找生产者要数据而是直接从容器里面取，这个容器就相当于一个缓冲区，平衡了生产者和消费者的处理能力，做到忙闲不均。中间的容器就是来对生产者和消费者做解耦的。

生产者消费者模型的优点在于**将生产者和消费者解耦**并且支持**多线程并发访问**，**支持忙闲不均**，而且消费者在拿到数据释放锁后在处理这个数据的时候，生产者也可以往容器里面生产数据。



基于阻塞队列的生产消费模型

阻塞队列 BlockingQueue:在多线程编程中，阻塞队列这种数据结构在实现生产者消费者模型中很常用。它与普通的队列区别在于，当队列为空时，从队列里面获取元素的操作会被阻塞，直到队列里被放的数据；当队列满时，往队列里放元素的操作也会被阻塞，直到有元素从队列中被取出。



C++ queue模拟阻塞队列的生产消费模型

我们用的条件变量是经过封装之后的：

```
1 // cond.hpp
2 #pragma once
3 #include <iostream>
4 #include <pthread.h>
5 #include "mutex.hpp"
6
7 class Cond
8 {
9 public:
10     Cond()
11     {
12         pthread_cond_init(&_cond, nullptr);
13     }
14     void wait(Mutex &mutex)
15     {
16         pthread_cond_wait(&_cond, mutex.Get());
17     }
18     void signal()
19     {
20         pthread_cond_signal(&_cond);
21     }
22     ~Cond()
23     {
24         pthread_cond_destroy(&_cond);
25     }
26
27 private:
28     pthread_cond_t _cond;
```

```
29 };
```

封装时不必将之前封装的 `Mutex` 引入成员变量，要将这两个模块解耦。

`Mutex` 和 `Cond` 基本上是一起创建的，将创建的 `Mutex` 传入 `Cond` 里面即可。这样可以使 `Cond` 更加具有通用性，可以传入其他类型的锁

```
1  // BlockQueue.hpp
2  #pragma once
3  #include "cond.hpp"
4  #include "mutex.hpp"
5  #include "thread.hpp"
6  #include <queue>
7  #include <iostream>
8
9  const int defaultnum = 10; // 默认队列长度
10
11 template <class T>
12 class BlockQueue
13 {
14 private:
15     bool IsFull()
16     {
17         return _q.size() == _cap;
18     }
19     bool IsEmpty()
20     {
21         return _q.empty();
22     }
23
24 public:
25     BlockQueue(int cap = defaultnum)
26         : _cap(cap),
27           _psleep(0),
28           _csleep(0)
29     {
30     }
31     void Equeue(const T &in) // 生产者调用，往队列里面生产东西
32     {
33         {
34             global_mutex gmutex(_mutex);
35             while (IsFull()) // 如果队列里为满，那就让线程一直
                               在条件变量下等待
```

```

36         {
37             _psleep++;
38             std::cout << "生产者进入等待" << std::endl;
39             Pcond.Wait(_mutex);
40             _psleep--;
41         }
42         // 队列有空位置
43         _q.push(in);
44         std::cout << "入队列" << std::endl;
45         if (_csleep > 0) // 通知消费者来消费
46         {
47             std::cout << "通知消费者来消费" <<
std::endl;
48             ccond.Signal();
49         }
50     }
51 }
52 T Pop() // 消费者调用
53 {
54     T out;
55     {
56         global_mutex gmutex(_mutex); // 利用类内的锁创建
可以自动解锁的锁
57         while (IsEmpty()) // 如果队列里为空，
那就让线程一直在条件变量下等待
58         {
59             _csleep++;
60             std::cout << "消费者进入等待" << std::endl;
61             Ccond.Wait(_mutex);
62             _csleep--;
63         }
64         out = _q.front();
65         _q.pop();
66         std::cout << "出队列" << std::endl;
67
68         if (_psleep > 0) // 通知生产者去生产
69         {
70             std::cout << "通知生产者去生产" <<
std::endl;
71             Pcond.Signal();
72         }
73     }

```

```

74         return out;
75     }
76     ~BlockQueue()
77     {
78     }
79
80 private:
81     std::queue<T> _q; // 可以用vector代替，都看心情~
82     int _cap;        // 队列容量
83
84     Mutex _mutex; // 为了维护p与c，p与p，c与c之间的互斥关系
85     Cond Pcond;   // 维护p与p之间的同步
86     Cond Ccond;   // 维护c与c之间的同步
87
88     int _psleep; // 生产者休眠数
89     int _csleep; // 消费者休眠数
90 };

```

为什么pthread_cond_wait需要互斥量

在使用条件变量（Condition Variable）时，`wait()` 和 `signal()` 操作必须在锁的保护下进行，这是由条件变量的核心设计目标决定的——**安全地等待和通知共享状态的变化**。以下是详细解释：

1. wait () 为什么要加锁？

(1) 原子性释放锁并进入等待状态

- `wait()` 的核心逻辑是：**释放锁** → **进入阻塞** → **被唤醒后重新获取锁**。
- 如果这个过程不是原子的，会导致竞态条件。例如：

```

1  # 错误示例（无原子性）
2  lock.release() # 释放锁
3  # 此时另一个线程可能修改共享状态并发出通知，但当前线程尚未阻塞
4  condition.wait() # 可能错过通知，永久阻塞

```

- **正确做法：**通过锁保证释放锁和阻塞操作的原子性，确保线程在释放锁后立即进入等待状态，不会错过其他线程的通知。

(2) 保护共享状态的可见性

- 线程在调用 `wait()` 前通常需要检查某个条件（如队列是否为空），这个检查必须在锁的保护下进行，以确保看到最新的共享状态。
- 示例：

```
1 with lock:
2     while not condition_met: # 在锁的保护下检查条件
3         condition.wait() # 原子释放锁并等待
4         # 条件满足后，自动重新获取锁，继续执行
```

2. `signal()`为什么要加锁？

(1) 确保通知操作的原子性

- `signal()` 操作需要修改条件变量的内部状态（如唤醒队列），如果多个线程同时调用 `signal()`，可能导致唤醒操作丢失或重复唤醒。
- **锁的作用**：保证 `signal()` 操作的原子性，避免竞态条件。

(2) 与 `wait()` 的锁保持一致

- 如果 `wait()` 和 `signal()` 使用不同的锁，会导致：
 - `wait()` 释放的锁与 `signal()` 操作的锁无关，无法正确同步。
 - 共享状态的修改和检查可能使用不同的锁，破坏一致性。

关键点：

- **生产者**在锁内修改队列并通知，确保消费者看到最新状态。
- **消费者**在锁内检查队列状态，若为空则原子释放锁并等待，被唤醒后重新获取锁继续执行。

条件变量使用规范

等待条件代码：

```
1 pthread_mutex_lock(&mutex);
2 while(条件为假)
3     pthread_cond_wait(cond, &mutex);
4 修改条件
5 pthread_mutex_unlock(&mutex);
```

给条件发送信号代码：

```
1 pthread_mutex_lock(&mutex);  
2 将条件变为真  
3 pthread_cond_signal(cond);  
4 pthread_mutex_unlock(&mutex);
```

POSIX信号量

POSIX信号量和SystemV信号量作用相同，用于同步操作，达到无冲突的访问共享资源的目的。但POSIX可以用于线程间同步。

初始化信号量

```
1 #include <semaphore.h>  
2 int sem_init(sem_t *sem, int pshared, unsigned int  
   value);  
3 参数:  
4     pshared: 0表示线程间共享，非零表示进程间共享  
5     value: 信号量初始值
```

销毁信号量

```
1 int sem_destroy(sem_t *sem);
```

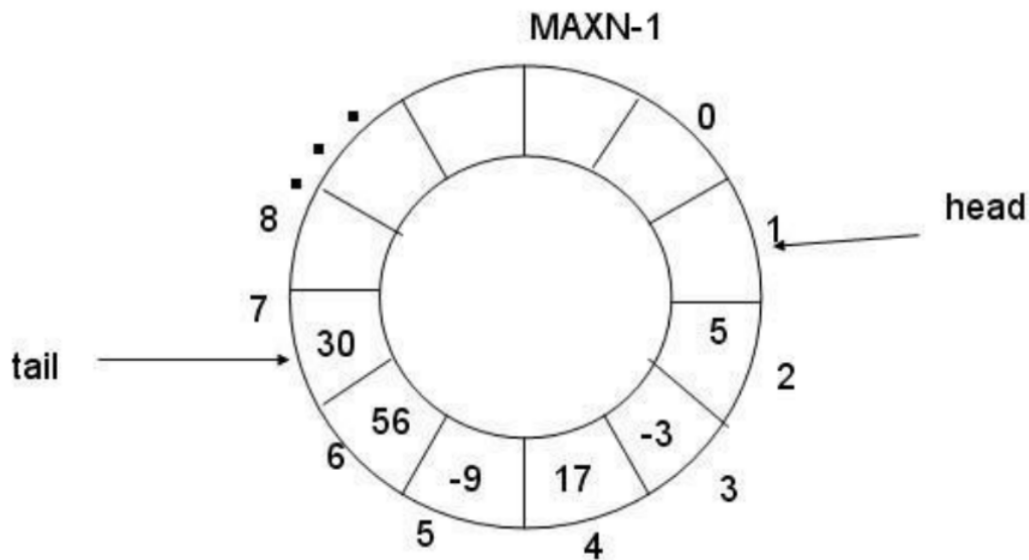
等待信号量

```
1 功能：等待信号量，会将信号量的值减一  
2 int sem_wait(sem_t *sem); // P操作
```

发布信号量

```
1 功能：发布信号量，表示资源使用完毕，可以归还资源了。将信号量加一  
2 int sem_post(sem_t *sem); // V操作
```


基于环形队列的生产消费者模型



环形结构的起始状态和结束状态都是一样的，不好判断为空，为满，所以可以通过加计数器或者标记位来判断满或者空。另外也可以预留一个空的位置，
`tail == head` 为空 `tail + 1 == head` 为满

现在我们可以用信号量，可以让生产者和消费者同时访问循环队列的不同位置，其本质就是一个计数器，实现多线程间同步，

封装信号量

```
1  #pragma once
2
3  #include <semaphore.h>
4
5  const int defalut = 1;
6  class Sem
7  {
8  public:
9      Sem(unsigned int sem_value = defalut)
10     {
11         sem_init(&_sem, 0, sem_value); // 第二个参数为零则在
线程间共享，第三个参数指定信号量的初始值
12     }
13     void P() // P(Proberen)操作 对信号量计数器--, 申请资源
14     {
```

```

15         sem_wait(&_sem); // 本身是原子的
16     }
17     void v() // v(verhogen)操作 对信号量计数器++, 释放资源
18     {
19         sem_post(&_sem); // 本身是原子的
20     }
21     ~Sem()
22     {
23         sem_destroy(&_sem);
24     }
25 private:
26     sem_t _sem;
27 };

```

需要注意的是对于生产者的资源就是空位置，对于消费者资源就是有资源的位置个数。另外为了支持多生产多消费，除了要靠信号量维持生产者和消费者之间的同步关系，还需要维持c与c之间和p与p之间的互斥关系，所以要再加两把锁。

```

1  #pragma once
2  #include "sem.hpp"
3  #include "mutex.hpp"
4  #include <vector>
5  #include <iostream>
6
7  const int default_len = 5; // 默认循环队列长度
8
9  template <class T>
10 class RingQueue
11 {
12 public:
13     RingQueue(int len = default_len)
14         : _rq(len),
15           _cap(len),
16           _c_step(0),
17           _data_sem(0),
18           _p_step(0),
19           _blank_sem(_cap)
20     {
21     }
22     void Equeue(T &in) // 生产者调用
23     {

```

```

24         _blank_sem.P(); // 申请资源
25     {
26         global_mutex glock(_p_lock); // 可以先让一批线程
        竞争信号量，再来竞争锁，后来的线程，就挂在信号量上
27         _rq[_p_step] = in;
28         _p_step++;
29         _p_step %= _cap;
30     }
31     _data_sem.v(); // 对消费者资源++;
32 }
33 T Pop() // 消费者调用
34 {
35     T data;
36     _data_sem.P(); // 申请资源
37     {
38         global_mutex glock(_c_lock);
39         data = _rq[_c_step];
40         _c_step++;
41         _c_step %= _cap;
42     }
43     _blank_sem.v(); // 对生产者资源++
44     return data;
45 }
46 ~RingQueue()
47 {
48 }
49
50 private:
51     std::vector<T> _rq; // 用vector来模拟循环队列
52     int _cap;           // 标记长度，用来模等，维持循环属性
53
54     int _c_step;        // 消费者位置
55     Sem _data_sem;      // 记录对于消费者来说的资源，也就是有数据的位置数
56
57     int _p_step;        // 生产者位置
58     Sem _blank_sem;     // 记录对于生产者来说的资源，也就是空位置数
59
60     // 为了支持多生产多消费，需要再加两把锁维持p与p，v与v之间的互斥关系
61     Mutex _c_lock;
62     Mutex _p_lock;

```

```
63  };
64
```

线程池

池化技术就是提前把需要用到的资源先申请到，放在一个数据结构里面管理起来（池子），在需要用到的时候拿出来用，不用了就放回池子里。这里的线程池，之前的进程池，项目中的高并发内存池，都是池化技术的体现。

日志与策略模式

计算机大佬们把一些常见的问题的解决方法归纳起来，总结成了各种解决方案，这就是**设计模式**。在接下来的简单日志系统中我们会用到**策略模式**

日志需要包含这几个元素：时间戳 日志等级 进程id 线程id 文件名 行号
日志内容，支持可变参数

```
1 [2025-05-05 23:45:25][DEBUG][2568001][Http.hpp][40]-
  _method: GET
2 [2025-05-05 23:45:25][DEBUG][2568001][Http.hpp][41]-_uri:
  ./wwwroot/index.html
3 [2025-05-05 23:45:25][DEBUG][2568001][Http.hpp][42]-
  _http_version: HTTP/1.1
4 [2025-05-05 23:45:25][INFO][2568001][Http.hpp][197]-client:
  39.144.218.179 : 2466
```

虽然有现成的方案 spdlog、glog、Boostlog、Log4cxx等，但是我们总得自己知道这个车是怎么跑起来的，所以我们还是要造一个轮子。

日志系统源码：<https://github.com/QinMou000/Code/tree/master/25/Log>

总结：

1. 这里我们用到的锁是自己之前封装好的，C++11中也给我们封装好了
`std::lock_guard<std::mutex> lock(_mutex);`
2. 首先我们定义一个基类，作为刷新策略的基类，之后再创建两个派生类，分别实现往显示屏和文件里面刷新日志的功能 `SyncLog`，显然这个部分会需要用到互斥锁。
3. 然后是一个 `logger` 类，里面包含一个我们之前定义的基类指针，我们通过改变这个指针的指向来改变日志的刷新策略。

4. 再 `Logger` 类里定义一个内部类 `LogMessage` 这个类用来生成一条完成的日志信息，由于要支持可变参数，所以在类里面需要我们实现一个 `operator<<` 运算符重载函数。
5. 我们用 `Logger` 类先定义一个全局对象，我们用这个全局对象中的 `operator()` 重载函数进行日志调用，这个重载函数里面是构造了一个临时内部类对象并进行返回，在进行返回的时候会自动调用析构函数，内部类里面包含了一个 `Logger` 引用，我们会在 `operator()` 中的参数传进去 `Logger` 的 `this` 指针，所以我们将这个内部类的析构函数就设计为根据 `Logger` 引用的刷新策略来进行刷新。
6. 我们的日志调用方法时通过一个宏来调用，这个宏里面就是一个 `operator()` 重载，我们只需要传日志等级和后面加上日志内容就行，文件名和行号用系统自带的宏。我们在设计一个获取时间戳的函数。改变日志刷新策略也是通过宏来调用 `log` 全局对象 `Logger` 类里面的函数

线程池设计

线程过多会带来调度开销，进而影响缓存局部性和整体性能。而线程池维护着多个线程，等待着监督管理者分配可并发执行的任务。这**避免了在处理短时间任务时创建与销毁线程的代价**。线程池不仅能够保证内核的充分利用，还能防止过分调度。可用线程数量应该取决于可用的并发处理器、处理器内核、内存、网络 `socket` 等的数量。

应用场景：

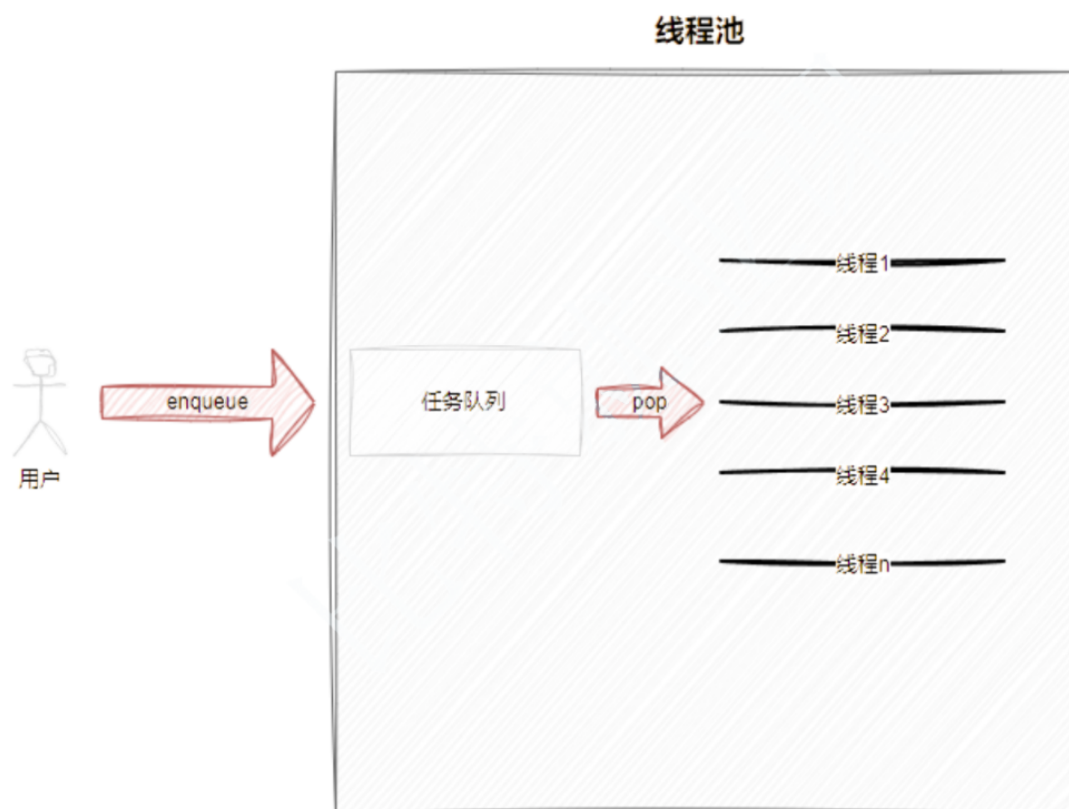
- 需要大量的线程来完成任务，且完成任务的时间比较短。比如web服务器完成网页请求则海洋的任务，使用线程池技术是非常合适的。因为单个任务小，点击次数也就是数量多。但对于长时间的任务，比如一个 `telnet` 的连接请求，线程池的优点就不明显了，`telnet` 会话时间往往远远大于创建和销毁线程的使时间。
- 对于性能要求苛刻的应用，快速相应等。
- 接受突发性的大量请求，但不至于使服务器因此产生大量线程的应用。突发性大量客户需求，在没有线程池的情况下，将产生大量线程，虽然理论上大部分操作系统线程数目最大值不是问题，短时间内产生的大量线程可能会使内存达到极限，出现错误。

线程池的种类

1. 创建固定数量线程池，循环从任务队列中获取任务对象，获取到任务对象后，执行任务对象中的任务接口

2. 浮动线程池，除了线程数量不固定，其他同上

这里我们选择固定线程个数的线程池



线程池源码：<https://github.com/QinMou000/Code/tree/master/25/ThreadPool>

总结：

- 这个线程池的类不算单例，我觉得重要的方法有以下几个
- 构造，当然是轮询新建几个线程放入线程队列里，将类中的 `handler` 方法传入，方便之后执行线程时线程从任务队列里面取任务
- `start`，轮询启动线程队列里的线程，这个很简单但很重要，没什么好说的
- `Handler` 和 `Equeue`，当线程池 `start` 之后，各个线程就执行到 `Handler` 里面了，线程就需要开始处理任务了，为了保证同步，需要互斥锁，在没有任务时，线程会都在条件变量下等，等到 `Equeue` 后任务队列被 `push` 任务后，某一个线程被唤醒，取出任务，执行任务
- **单例模式**，为了实现单例模式，我们首先需要将构造函数和 `start` 私有，此外，类中还需要一个单例指针，这个指针是 `static` 的，也就是说整个文件只有这一个指针，默认为空，当有线程第一次调用 `GetInstance` 时，我们 `new` 一个线程池对象给这个单例指针。当然这

其中要保证在多线程调用时不会出现错误。所以保证同步，还需使用一个互斥锁，也是 `static` 的

线程安全的单例模式

用一个洗碗的例子来形象的解释饿汉和懒汉

- 饿汉：吃完饭立刻洗碗，下一顿时可以直接拿着碗就吃饭
- 懒汉：吃完饭不着急洗碗，等到下一顿饭要用这个碗再洗碗

饿汉方式和懒汉方式实现单例

饿汉

```
1  template <typename T>
2  class Singleton {
3      static T data; // 类中本来就有这样一个数据，你要直接给你
4  public:          // 只要通过Singleton这个包装类来使用T对象，则
                  // 一个进程中只有一个T对象实例
5      static T* GetInstance() {
6          return &data;
7      }
8  };
```

懒汉

```
1  template <typename T>
2  class Singleton {
3      static T* inst; // 类中只有这个类类型的指针，在你需要的时候
                     // 再去new
4  public:
5      static T* GetInstance() {
6          if (inst == NULL) {
7              inst = new T();
8          }
9          return inst;
10     }
11 };
```

这样的方式明显存在线程安全问题，第一次调用 `GetInstance` 时，如果两个线程同时调用，可能会 `new` 出两个 `T` 对象实例，所以需要锁

```

1 // 懒汉模式，线程安全
2 template <typename T>
3 class Singleton {
4     volatile static T* inst; // 需要设置 volatile 关键字，否则可能被编译器优化
5     static std::mutex lock;
6 public:
7     static T* GetInstance() {
8         if (inst == NULL) { // 双重判定空指针，降低锁冲突的概率，提高性能
9             lock.lock(); // 使用互斥锁，保证多线程情况下也只调用一次 new
10            if (inst == NULL) {
11                inst = new T();
12            }
13            lock.unlock();
14        }
15        return inst;
16    }
17 };

```

线程安全和重入问题

概念

线程安全：就是多个线程在访问共享资源时能够正确的执行，不会相互干扰或破坏彼此的执行结果。一般而言，多个线程并发同一段只有局部变量的代码时，不会出现不同的结果。但是对全局变量或者静态变量进行操作，并且没有锁保护的情况下，容易出现该问题

重入：同一个函数被不同的执行流调用，当前执行流还没有执行完，就有了其他执行流再次进入，称为重入。一个函数在重入的情况下，运行结果不会出现任何不同或者问题，称该函数为可重入函数，否则称为不可重入问题。

根据现在的理解将重入分为两种情况

1. 多线程重入
2. 当程序执行时，若收到一个信号（如键盘中断 `SIGINT`），系统会暂停当前执行流，转而执行信号处理函数。如果信号处理函数中调用了当前正在执行的函数，就会导致**信号重入**（同一执行流被中断后再次进入同一函数）。

常见线程不安全的情况

- 不保护共享变量的函数
- 函数状态随着被调用发生变化的函数
- 返回指向静态变量指针的函数
- 调用线程不安全的函数

常见不可重入的情况

- 调用了malloc/free函数，malloc是使用全局链表来管理堆的
- 调用了标准I/O库的很多实现都以不可重入的方式使用全局数据结构
- 可重入函数内部使用了静态的数据结构

函数是可重入的，那就是线程安全的；如果函数是线程安全的，不一定是可重入的。

常见锁概念

死锁

死锁是指在一组进程中的各个进程均占有不会释放的资源，但因互相申请被其他进程占用不会释放的资源而处于的一种永久的等待状态

方便描述，假设线程A，线程B必须同时持有锁1和锁2才能访问后续资源

申请一把锁是原子的，但是同时申请两把锁就有可能，A线程申请了锁1，B线程申请了锁2，他们谁也不让着谁，就造成了死锁

死锁的四个必要条件

- 互斥条件：一个资源每次只能被一个执行流使用
- 请求与保持条件：一个执行流因请求资源而阻塞时，对已获得的资源不释放
- 不剥夺条件：一个执行流已获得的资源，在未使用完之前不能强行剥夺
- 循环等待条件：若干执行流之前形成一种头尾相接的循环等待资源关系

避免死锁

- 破坏循环等待条件问题：资源一次性分配，使用超时机制、加锁顺序一致、避免锁未释放的场景

```
1  #include <iostream>
2  #include <mutex>
3  #include <thread>
4  #include <vector>
5  #include <unistd.h>
6  // 定义两个共享资源（整数变量）和两个互斥锁
7  int shared_resource1 = 0;
8  int shared_resource2 = 0;
9  std::mutex mtx1, mtx2;
10 // 一个函数，同时访问两个共享资源
11 void access_shared_resources()
12 {
13     // std::unique_lock<std::mutex> lock1(mtx1,
14     std::defer_lock);
15     // std::unique_lock<std::mutex> lock2(mtx2,
16     std::defer_lock);
17     // // 使用 std::lock 同时锁定两个互斥锁
18     // std::lock(lock1, lock2);
19     // 现在两个互斥锁都已锁定，可以安全地访问共享资源
20     int cnt = 10000;
21     while (cnt)
22     {
23         ++shared_resource1;
24         ++shared_resource2;
25         cnt--;
26     }
27     // 当离开 access_shared_resources 的作用域时，lock1 和
28     lock2 的析构函数会被自动调用
29     // 这会导致它们各自的互斥量被自动解锁
30 }
31 // 模拟多线程同时访问共享资源的场景
32 void simulate_concurrent_access()
33 {
34     std::vector<std::thread> threads;
35     // 创建多个线程来模拟并发访问
36     for (int i = 0; i < 10; ++i)
37     {
```

```

35         threads.emplace_back(access_shared_resources);
36     }
37     // 等待所有线程完成
38     for (auto &thread : threads)
39     {
40         thread.join();
41     }
42     // 输出共享资源的最终状态
43     std::cout << "Shared Resource 1: " << shared_resource1
44     << std::endl;
45     std::cout << "Shared Resource 2: " << shared_resource2
46     << std::endl;
47 }
48 int main()
49 {
50     simulate_concurrent_access();
51     return 0;
52 }

```

`std::defer_lock` 表示创建锁对象时不立即加锁。

`std::lock(lock1, lock2)` 以原子方式同时锁定多个锁，避免死锁。

```

1 $ ./a.out // 不一次申请
2 Shared Resource 1: 94416
3 Shared Resource 2: 94536

```

```

1 $ ./a.out // 一次申请
2 Shared Resource 1: 100000
3 Shared Resource 2: 100000

```

STL、智能指针的线程安全

STL中的容器不是线程安全的，因为STL设计初衷是将性能发挥到极致

`unique_ptr` 由于只在当前代码块范围内生效，所以不涉及线程安全问题

`shared_ptr` 多个对象需要共用一个引用计数，所以是存在线程安全问题的，这个在设计 `shared_ptr` 的时候也考虑到了，基于原子操作 `CAS` 的方式保证了 `shared_ptr` 能够高效、原子的操作引用计数

1. **原子操作**：使用 `std::atomic` 确保引用计数的增减操作是原子的。

2. **控制块唯一性**：通过 `std::call_once` 或等价机制确保同一对象的所有 `shared_ptr` 共用同一个控制块。
3. **安全的共享接口**：提供 `std::enable_shared_from_this` 确保对象能安全获取自身的 `shared_ptr`。